Test Plan Humming bird
Document Information

- **Test Plan ID**: HBTP01
  Hummingbyrdinc.com
- **Version:1.02**
- **Date: 10-08-2024**
- **Last Updated :9-09-2023**
- **Approved By : Chandra Shekhar**
- **Date: 20-09-2025**

Purpose

The purpose of this test plan is to ensure that the *Personality Grooming Website* meets its functional, usability, security, performance, and reliability requirements, and provides an excellent user experience to its target audience.

Specifically, through manual testing we aim to:

1. **Verify core functionality**: Ensure that all key features (e.g., user registration/login, profile creation, course browsing and enrollment, booking or scheduling consultations, payment/checkout, content access) work correctly as intended.
2. **Ensure usability & user experience**: Confirm that navigation is intuitive, content (text, images, video) displays correctly, forms are easy to fill, and that the website is accessible and responsive across devices (desktop, tablet, mobile).
3. **Validate performance & responsiveness**: Ensure pages load within acceptable times, interactive elements respond smoothly, and no major lag or jank in essential workflows.
4. **Ensure security & data protection**: Make sure personal user data (profiles, payment data, etc.) is handled safely, stored or transmitted securely, that there is protection against common vulnerabilities (e.g. SQL injection, XSS, etc.), and that the authentication flow is robust.
5. **Check compatibility**: Ensure that the site works across major browsers, operating systems, and device types that are expected among the users.
6. **Prevent regressions**: Ensure that as new features are added or changes are made, previously working functionality continues to work seamlessly.
7. **Support business goals**: Ensure that the site helps achieve strategic goals like sign-ups, conversion rate in course bookings, customer satisfaction, retention, etc., by delivering a stable, trustworthy, and pleasant user experience.

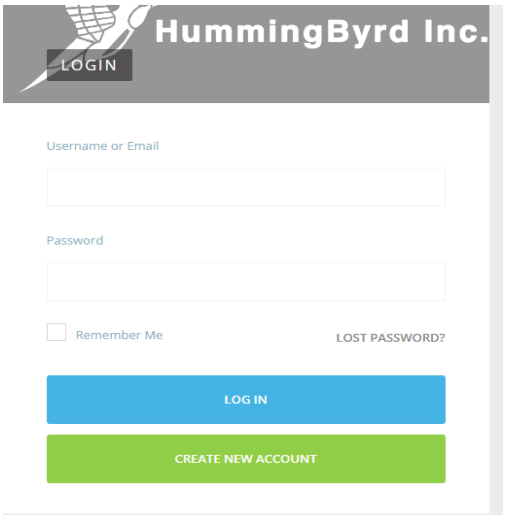
Scope

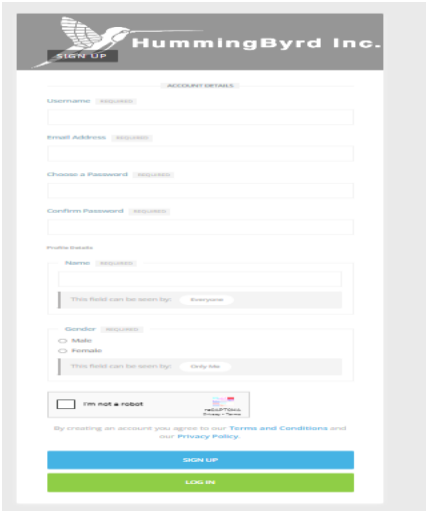
1. Features / Functionality In Scope

These are all the features that will be verified during the test plan:

- **User Management**

- Registration / signup (valid and invalid data)
- Login / Logout functionality
- Password reset / Forgot password flows
- Profile creation & update (uploading photo, bio, interests)





- **Course / Learning Content**
  - Browsing catalog of courses (filters, search)
  - Viewing course details (syllabus, instructor info, prerequisites)
  - Enrolling in courses
  - Accessing content: videos, PDFs, quizzes, assignments
- **Payments & Transactions**
  - Payment gateway integration (credit/debit, UPI, wallets, etc.)
  - Handling of payment failures, receipts, refunds if applicable
- **Booking / Mentorship / Consultation**
  - Scheduling sessions with mentor/coach
  - Calendar integration / availability management
  - Notifications / reminders
- **Blog, Articles & Media Content**
  - Reading, searching, filtering articles/blogs
  - Video streaming or embedded video content playback
  - Comments / Reviews (if present)
- **User Interactions & Community Features** *(if present)*
  - Discussion forums / comments
  - Social sharing
  - Ratings / feedback
- **Non-functional Aspects**
  - Usability: intuitive UI / navigation, form validations
  - Responsiveness: mobile, tablet, desktop views

    - Performance: page load times, video playback smoothness
    - Security: authentication, data privacy, secure transmission
- **Compatibility**
    - Across major browsers (Chrome, Firefox, Safari, Edge)
    - Across common OS & devices (Windows, Mac, iOS, Android)
    - Different screen resolutions / viewports

---

2. Features / Functionality Out of Scope

These are explicitly excluded during this test plan's duration:

- Admin back-end dashboard (if built separately) unless agreed
- Internationalization / localization (if the site is only in one language) unless specified
- Offline version / mobile app (if only the web portal is being tested)
- Video content authoring / uploading flows (if those are handled by content team separately)
- Advanced accessibility compliance beyond basic WCAG checks (unless required)
- Load or stress testing beyond nominal user load (unless specifically scheduled)

---

3. Types of Testing In Scope

- Functional testing of all in-scope features
- Usability / UI-UX testing
- Responsiveness / layout testing
- Security basic tests (authentication, data encryption, etc.)
- Compatibility / cross browser and device testing

---

4. Types of Testing Out of Scope

- Deep penetration testing or security audits by external parties (unless contracted)
- Very high load, stress or spike testing beyond standard usage scenario
- Long-term stability (e.g., running sessions for days) unless agreed

---

5. Constraints / Dependencies

- The project must have test data (user accounts, payment credentials) ready
- Dependent services (payment gateway, video streaming servers) must be available in test / staging environment
- Content for courses/blogs/videos must be fully populated or mocked before testing
- Devices & browsers must be made available in test environment

Test Environment Section

This section describes the environment(s) needed so that testing is accurate, consistent, and reflects as close as possible the real (production) setup. It covers hardware, software, network, data, tools, and configurations.

---

1. Environments Needed

Environment	Purpose	When Used
Development / Local	For developers to build and unit-test features on their machines. Not stable; rapid changes.	During feature development & early bug fixes.
QA / Test / Integration	A shared environment for QA to run functional, integration, UI, usability testing. Mirrors production in key software components.	After features complete dev stage, before staging.
Staging / Pre-Production	As close to production as possible: same configuration, similar data, same 3rd party integrations. Used for final acceptance, regression, performance sanity, UAT.	Before final sign-off & deployment to production.
Production / Live	The live site used by end users. We try to avoid testing here except for limited smoke checks or monitoring.	After deployment; more for monitoring, hotfixes.

---

2. Hardware & Infrastructure

- **Servers / Compute**
  - Web server(s) (frontend)
  - Application server(s) / backend APIs / services
  - Database server(s)
  - File storage servers or equivalent (if hosting media, video, images)
- **Network**
  - Bandwidth, latency similar to production where possible
  - DNS setup: separate domains/subdomains for staging vs production
- **Other Services**
  - Caching servers (Redis, Memcached)
  - Search / indexing services (Elasticsearch, etc.)
  - Messaging/notification services (email, SMS, push)
  - Payment gateway sandbox / mock services
  - Media streaming / video services (or mocks)

3. Software & Configurations

- **Operating Systems** & versions matching production.
- **Web Server Software** (e.g. Nginx, Apache) same version/config.
- **Application Stack**: language runtime, framework version, libraries, SDKs identical or nearly identical to production.
- **Database**: same type (MySQL / PostgreSQL / MongoDB etc.), schema the same. Possibly, test database should be a copy of production (sanitized) or with representative data.
- **Environment Variables / Config Files**:
  - Config for different environments (e.g. API endpoints, keys, feature flags)
  - Secrets / credentials (use sandbox credentials for payment etc., not real ones)
- **Third-party Integrations / APIs**:
  - Where possible, use testing/sandbox versions of external APIs (e.g. payment gateways, email/SMS providers)
  - For services that don't offer sandbox, consider mocking responses

---

4. Data Requirements

- **Test Data**: Realistic but not sensitive; if using production-derived data, must be anonymized.
- **Accounts**: Different user types (normal users, premium, mentors/coach/admin etc.) with valid credentials for testing.
- **Media / Content**: courses, blogs, videos populated sufficiently.
- **Edge Cases Data**: invalid inputs, boundary values, malformed data.

---

5. Tools & Access

- **Bug / Defect Tracking Tool** (e.g. JIRA, Trello, etc.)
- **Test Case Management Tool** (or simple repository)
- **Version Control System** (Git) to pull correct builds
- **Deployment / CI/CD** tools (so testers know which build/version is deployed)
- **Monitoring / Logs Access** for test environment (to inspect errors)
- **Browser / Device Lab** for cross-browser/device testing (desktop, tablet, mobile)

---

6. Environment Configuration & Setup

- **Domain / URLs**: staging.example.com, qa.example.com etc.

- **SSL / HTTPS** configured similar to production.
- **User Permissions**: testers, admins, mentors etc.
- **Clean state / Reset**: ability to reset state between test runs (clear DB, cache etc.)
- **Logging / Debugging** enabled (but ensure logs don't expose sensitive data)

---

7. Non-Functional Environment Needs

- **Performance Testing**: environment capable of simulating load; CPU, memory, network similar to production.
- **Usability / Responsiveness**: real devices, different screen sizes.
- **Security Testing**: environment configured so that authentication, encryption, data handling are same as production.

---

8. Constraints / Risks Related to Environment

- May not have exact hardware parity → some performance differences.
- Some third-party sandbox services might behave differently from production.
- Maintaining test/staging data may be time consuming.
- Environment downtime or misconfiguration can block testing.

1. **Defect Reporting Process**
    - During manual testing (functional, UI, content, booking flows, payment flows), testers immediately log defects in the tracking tool.
    - All relevant information must be filled before the bug is assigned.
    - The lead developer or module owner should review newly reported bugs within *24 hours*.
    - Once a fix is deployed in the test/staging environment, tester retests; if behavior matches expected result, bug is marked *Closed*, else *Reopened*.
2. **Bug Report Fields / Template**

Field	Description
Bug ID	Auto-generated / custom ID for traceability.
Title	e.g. "Course Enrollment fails when UPI payment selected"
Reporter	Tester's name or ID.
Date Reported	Date / Time stamp.
Environment / Version	e.g. Staging v2.3.1; Browser (Chrome 125), Device (Android phone)
Module / Feature	e.g. Registration / Profile, Courses, Booking, Payments, Blog, Video Content

Field	Description
Steps to Reproduce	Step by step to reproduce (including data, user type).
Test Data	e.g. sample user credentials, payment method, video file etc.
Expected Result	What should have happened (e.g. “Transaction completes, receipt shown, access to course unlocked”)
Actual Result	What happened (error message, crash, etc.)
Severity	(Blocker / Critical / Major / Minor / Cosmetic)
Priority	High / Medium / Low
Attachments	Screenshots / Error logs / Payment gateway response / Video or screen recording where relevant.
Status	(New / Assigned / Fixed / Retest / Verified / Closed / Reopened / Deferred)
Comments / Notes	Workaround, whether it impacts end-users, if it’s reproducible always or only in certain browsers/devices etc.

3. **Severity & Priority Definitions**
   Same as above, but map some examples:

- **Blocker**: Booking/Payment system completely down; users cannot enroll in any course.
- **Critical**: Profile photo upload crashes browser; major feature inaccessible.
- **Major**: Payment failure for certain payment methods; booking calendar not showing correct slots.
- **Minor**: Blog post alignment issue; minor typo in content.
- **Cosmetic**: Small UI border overflow; non-critical visual glitch.

Priority likewise:

- **High**: Must fix before next public release / deployment.
- **Medium**: Fix for upcoming sprint.
- **Low**: If time permits; cosmetic or minor.

4. **Defect Life-Cycle & Status**

- New → Assigned → In Progress → Fixed → Retest → Verified → Closed
- Possible statuses: Reopened (if fix fails), Deferred (if decided to postpone), Rejected (if not a bug or out of scope)

5. **Roles & Responsibilities**

Role	Responsibility
QA Tester	Discover and log bugs with all detail; retest; verify closure.
Developer / Module Owner	Analyze & fix; communicate when fix is ready; update status.

Role	Responsibility
QA Lead / Test Lead	Monitor bug reports, ensure correct classification; ensure priority assignments; ensure timely retest.
Product Owner / Manager	Make decisions on priority vs release timeline; approve fixes for critical bugs.

6. **Defect Tracking Tool**

- Tool: e.g. JIRA (or your chosen)
- All testers, devs, QA lead have access.
- Notifications should be enabled on status changes, assignments.

7. **Metrics & Reporting**

- Weekly bug summary by severity and module (courses, payments, bookings, etc.)
- Number of open bugs vs closed bugs per sprint / test run
- Average time to fix & retest
- Number of reopened bugs
- Aging: count of bugs open more than X days (e.g., more than 3 days for Critical)

8. **Escalation Criteria**

- Any *Blocker* bug not resolved within **24 hours** in staging.
- If bugs block test progress for other features.
- If number of open Critical or Blocker bugs exceeds *threshold* (e.g. >3)

9. **Review & Closure Criteria**

- Tester verifies fix in the same environment where bug was found (or noted environment).
- Expected behavior matches documented expected result.
- Any regression caused by fix must be checked.
- If bug is rejected / deferred, document reasoning.
-

Test Strategy

- **Objectives**

- Ensure that users can register, log in, create/update profile, and access all core features (courses, mentor booking) without major defects.
- Ensure payment flows work reliably (including handling failures & refunds).
- Provide smooth experience on mobiles & tablets (responsive UI).
- Ensure trust & security of user data.
- Deliver acceptable performance (pages load quickly; video content streams smoothly).

- **Test Types / Levels**

    - *Functional Testing*: Registration/login, profile, course enrollment, payment, booking sessions.
    - *Integration Testing*: Course modules with video service, payment gateway, mentor availability.
    - *System / End-to-End Testing*: Full user journeys ("new user signs up → enrolls in course → books consultation → makes payment").
    - *UAT / Acceptance*: Stakeholders test delivered features before release.
    - *Non-Functional*: Performance, security, compatibility, usability.

- **Manual vs Automation**

    - Manual testing for new features, exploratory testing, usability and UI issues.
    - Automated testing for regression, smoke tests (e.g. login, critical user flows), and repeatable tests (form submission, payment success/fail paths).
    - Use automation tools/frameworks (e.g. Selenium / Cypress / Playwright) if available.

- **Risk & Prioritization**

    - High-risk areas: payment gateway, booking scheduling (availability conflicts), user data privacy/security.
    - Also course content delivery (video streaming reliability).
    - Prioritize test cases that affect revenue or user trust (e.g. payment failures, booking errors).

- **Test Environment Strategy**

    - Environments: QA (for development builds), Staging (production-like with mocked/sandbox services), Device/Browser Lab.
    - Use sandbox versions of payment gateways; mock external services where needed.
    - Test across Chrome, Firefox, Safari, Edge; on iOS & Android; desktop & mobile views.

- **Entry & Exit Criteria**

    - **Entry**: Feature development complete; code review done; environment stable; test data ready; build deployed to test/staging.
    - **Exit**: All *critical* & *high severity* bugs fixed/retested; no blocker defects; acceptance from stakeholders; regression suite passed.

- **Regression Strategy**

    - Maintain a core regression suite covering key user journeys (login, enrollments, payments, bookings).
    - Run regression suite on each release candidate.
    - Also regression run after patches/fixes.

- **Non-Functional Testing Strategy**

  - **Performance**: Measure page load times; video buffer times; check behavior under moderate concurrent usage.
  - **Security**: Basic penetration tests, input validation, secure storage of user data, SSL/TLS.
  - **Usability**: Review navigation flows; form feedback; content readability; mobile usability.
  - **Compatibility**: As above with devices/browsers.

- **Defect / Bug Process & Reporting**

  - Use bug-tracking tool (e.g. JIRA / similar) to log defects.
  - Include severity & priority; attachments; environment details.
  - Triage meetings for high/critical bugs.
  - Retest when fixes are deployed; re-open if not fixed.

- **Tools, Resources & Structure**

  - Tools: Test case management (manual), automation tool for regression, bug tracker, possibly tools for performance testing (e.g. load testing tools), video streaming monitoring.
  - Roles: QA testers, automation engineers (if any), developers, product owner for acceptance; UI/UX designer input for usability.
  - Ensure testers have domain knowledge (personality grooming, course content).

- **Schedule / Milestones**

  - Feature development → QA testing → staging validation → UAT → Release.
  - Regression scheduled just before staging and pre-release.
  - Booking / payment critical features validated early.

- **Contingency / Mitigation Plans**

  - If staging environment has downtime, fall back to QA environment where possible.
  - If payment sandbox unreliable, use mocks / test hooks.
  - If video streaming issues arise late, revert to lower resolution temporarily.
  - If resource constraints (e.g. tester shortage), deprioritize low severity/cosmetic tests.

Test Schedule

Phase / Activity	Start Date	End Date	Duration	Responsible	Notes / Dependencies
Test Planning & Kick-off	2025-10-01	2025-10-03	3 days	QA Lead / Project Manager	Confirm requirements, setup test plan document
Environment Setup	2025-10-04	2025-10-06	3 days	DevOps / QA Team	Setup QA & Staging environments; payment

Phase / Activity	Start Date	End Date	Duration	Responsible	Notes / Dependencies
					gateway sandbox, video service mock
Test Case & Data Preparation	2025-10-07	2025-10-13	7 days	QA Team	Create test cases for all in-scope features; prepare test data (users, content, media)
Smoke / Sanity Testing	2025-10-14	2025-10-15	2 days	QA Team	Verify core flows (login, course access, payment gateway basic) in staging/QA
Functional Testing - Phase 1	2025-10-16	2025-10-23	8 days	QA Team	Test user management, course catalog, enrollment, profiles etc.
Functional Testing - Phase 2 (Booking, Payments, Notifications)	2025-10-24	2025-10-30	7 days	QA / Dev Team	Test booking/mentor sessions, payment success/failures, email / reminders
UI / Usability / Responsiveness Testing	2025-10-26	2025-10-31	6 days	QA / UX Designer	Across devices & browsers; mobile / tablet / desktop views
Security / Basic Vulnerability Testing	2025-10-28	2025-11-01	5 days	Security Specialist / QA	Test authentication, data encryption, input validation etc.
Regression Testing	2025-11-02	2025-11-04	3 days	QA Team	Re-run core user journeys after fixes from earlier phases
Performance / Load Testing (if applicable)	2025-11-05	2025-11-07	3 days	QA / DevOps	Check page load, video streaming, concurrent users etc.
UAT / Stakeholder Review	2025-11-08	2025-11-10	3 days	Stakeholders / Product Owner / QA	Review features, get sign-off, feedback
Bug Fix & Retest	2025-11-11	2025-11-13	3 days	Dev / QA	Fix high/critical bugs & verify them

Phase / Activity	Start Date	End Date	Duration	Responsible	Notes / Dependencies
Final Regression & Sanity	2025-11-14	2025-11-15	2 days	QA Team	Confirm that production-like build is stable
Test Closure & Reporting	2025-11-16	2025-11-17	2 days	QA Lead	Prepare test closure report, lessons learned, open issues etc.

Entry & Exit Criteria

STLC Phase	Entry Criteria	Exit Criteria
Requirement Analysis	• Requirements document (functional + non-functional) is available • Acceptance criteria defined • Application architecture / high level design available • Stakeholders / BAs accessible for clarifications • Assumptions & constraints documented	• Requirement Traceability Matrix (RTM) drafted and reviewed • Automation feasibility report (if applicable) • Ambiguities / gaps in requirements resolved or logged • Sign-off on analyzed requirements
Test Planning	• Requirements + RTM available • Automation feasibility (if needed) • Project constraints / assumptions known • Resource availability known	• Test plan & test strategy document prepared & reviewed • Effort estimation and schedule approved • Risks, dependencies, entry/exit criteria documented • Tools, roles & responsibilities decided • Stakeholder sign-off on plan
Test Case Development / Design	• Test plan & strategy approved • Requirements / RTM stable • Automation plan / scripts direction known	• Test cases / test scripts developed & reviewed • Test data prepared • Test cases mapped to requirements (via RTM) • Reviews completed and baselined
Test Environment Setup	• System design / architecture available • Environment setup plan / configuration details known	• Environment installed & configured • Test data loaded • Smoke / sanity test passed (basic sanity of environment)

STLC Phase	Entry Criteria	Exit Criteria
	• Hardware & software requirements identified	• Environment stability verified • Access rights / credentials established
Test Execution	• Test environment is ready • Test cases / scripts & test data available • Build under test deployed • Smoke test passed to confirm basic readiness	• All planned test cases executed or appropriately blocked • Defects logged & tracked • Retesting and regression done for fixed defects • Exit metrics / test summary prepared • All critical / high severity defects resolved (or accepted with risk)
Test Cycle Closure	• Test execution completed • Defect reports available • Exit metrics & summary ready	• Test closure report signed off • Lessons learned / retrospective documented • All test artifacts archived • Outstanding (deferred) defects listed with status • Stakeholder sign-off for closure